Package Plan: canopy/dnd

Ambition

canopy/dnd aims to be the **best drag-and-drop library in any ecosystem** -- not just the Elm/Canopy world, but surpassing React solutions like Atlassian's Pragmatic Drag and Drop, dnd-kit, and the legendary react-beautiful-dnd. This means:

- Every use case that dnd-kit/Pragmatic DnD can handle, we handle -- plus the ones they can't
- The animation quality of react-beautiful-dnd (spring physics, weighted drops) -- but with better performance
- The accessibility rigor of React Aria (Adobe) -- full input parity, not an afterthought
- First-class support for complex patterns: kanban boards, page builders, tree editors, dashboard layouts, nested sortables, multi-select drag

Motivation

Drag-and-drop is fundamental to modern UIs: kanban boards, sortable lists, file uploads, layout builders. Existing solutions in the Elm ecosystem (annaghi/dnd-list) only handle sortable lists with mouse events and lack touch support, keyboard accessibility, and cross-container dragging.

The HTML5 Drag and Drop API is notoriously inconsistent across browsers and does not work on touch devices. The pointer events approach (as proven by dnd-kit) provides a unified input model across mouse, touch, and pen, with full control over drag previews and behavior.

canopy/dnd aims to be a complete, accessible drag-and-drop library for Canopy applications, inspired by dnd-kit's layered architecture and annaghi/dnd-list's TEA-native approach.

Design Principles

1. **TEA-native**: All state managed through Model/update/view, no hidden mutable state
2. **Pointer events over HTML5 DnD**: Unified mouse/touch/pen support, no polyfills needed
3. **Accessible by default**: Keyboard reordering and ARIA live announcements built in
4. **Composable**: Small building blocks that compose into complex interactions
5. **Zero DOM requirements**: Works with any HTML structure, no wrapper divs forced on users
6. **Physics-based motion**: Spring animations for drops and displacement, not linear/ease-in-out
7. **Scale to thousands**: Virtualization-aware from day one, 60fps with 1000+ items
8. **RTL-native**: Right-to-left layout support is not an afterthought

Dependencies

- `canopy/core` (basics, platform)
- `canopy/html` (view rendering, events)
- `canopy/json` (event decoder)
- `canopy/browser` (subscriptions for pointer events, keyboard events, animation frames)

Module Structure

```
src/
  DnD.can                    -- Main API, context, configuration
  DnD/
    -- Core (v1 - implemented)
```

```
    Draggable.can             -- Making elements draggable
    Droppable.can             -- Defining drop zones
    Sortable.can              -- High-level sortable list API
    Sensor.can                -- Input sensors (pointer, keyboard)
    Collision.can             -- Collision detection algorithms
    Modifier.can              -- Transform modifiers (axis lock, snap, bounds)
    Announcements.can         -- ARIA live region announcements
    AutoScroll.can            -- Auto-scroll near container edges

    -- Advanced modules (v2)
    Tree.can                  -- Tree drag-and-drop (file explorers, nested menus)
    Kanban.can                -- High-level kanban board with columns + cards
    Grid.can                  -- Dashboard/grid layout with resize + drag
    MultiSelect.can           -- Multi-item selection and group drag
    Handle.can                -- Drag handle separation from draggable container
    Animation.can             -- Spring physics, drop curves, displacement easing
    Hitbox.can                -- Edge proximity detection (closest edge of target)
    Monitor.can               -- Global drag operation observers
    Operation.can             -- Drag operation types (move, copy, link)
    FileDrop.can              -- OS file drop zones (DataTransfer API)
    Virtualized.can           -- Integration with virtualized/windowed lists
    History.can               -- Undo/redo stack for drag operations
    Testing.can               -- Test utilities for simulating drag in tests
```

Types

DnD (Main Module)

```
-- Opaque type holding all drag-and-drop state
type State id

-- Configuration for the drag-and-drop context
type alias Config id msg =
    { onDragStart : DragStart id -> msg
    , onDragOver : DragOver id -> msg
    , onDragEnd : DragEnd id -> msg
    , onDragCancel : msg
    , sensors : List (Sensor id)
    , collisionDetection : CollisionDetection id
    , modifiers : List (Modifier id)
    , announcements : Announcements id
    , autoScroll : AutoScrollConfig
    }

-- Events emitted during drag lifecycle
type alias DragStart id =
    { active : DragItem id
    }

type alias DragOver id =
    { active : DragItem id
```

```
    , over : Maybe (DropTarget id)
    , delta : Delta
    }

type alias DragEnd id =
    { active : DragItem id
    , over : Maybe (DropTarget id)
    }

-- Core data types
type alias DragItem id =
    { id : id
    , rect : Rect
    , data : Dict String String
    }

type alias DropTarget id =
    { id : id
    , rect : Rect
    , data : Dict String String
    }

type alias Delta =
    { x : Float
    , y : Float
    }

type alias Rect =
    { x : Float
    , y : Float
    , width : Float
    , height : Float
    }

type alias Point =
    { x : Float
    , y : Float
    }

-- The active drag operation, if any
type DragOperation id
    = NotDragging
    | Dragging (ActiveDrag id)

type alias ActiveDrag id =
    { item : DragItem id
    , current : Point
    , delta : Delta
    , over : Maybe (DropTarget id)
    }
```

DnD Function Signatures

```
-- Initialize drag-and-drop state
init : State id

-- Update the drag-and-drop state (call from your update function)
update : Config id msg -> Msg id -> State id -> ( State id, Cmd msg )

-- Internal message type (opaque, users pattern match on Config callbacks)
type Msg id

-- Subscriptions needed for drag tracking (pointer move, pointer up, keyboard)
subscriptions : Config id msg -> State id -> Sub msg

-- Query the current drag operation
activeDrag : State id -> Maybe (ActiveDrag id)

-- Check if a specific item is being dragged
isDragging : id -> State id -> Bool

-- Get the transform delta for a dragged item
dragDelta : State id -> Maybe Delta
```

DnD.Draggable

```
-- Attributes to apply to a draggable element
type alias Attributes msg =
    { listeners : List (Html.Attribute msg)
    , styles : List (Html.Attribute msg)
    , attrs : List (Html.Attribute msg)
    }

-- Make an element draggable
draggable : Config id msg -> State id -> id -> Attributes msg

-- Apply drag transform to the element being dragged (translate via CSS transform)
dragStyles : State id -> id -> List (Html.Attribute msg)

-- Associate arbitrary data with a draggable item
withData : List ( String, String ) -> Attributes msg -> Attributes msg

-- Render a custom drag overlay instead of moving the original element
type alias DragOverlay msg =
    { view : DragItem id -> Delta -> Html msg
    }

-- View the drag overlay (rendered at the document level, follows pointer)
viewOverlay : Config id msg -> State id -> (DragItem id -> Delta -> Html msg) ->
Html msg
```

DnD.Droppable

```
-- Attributes to apply to a drop zone
type alias Attributes msg =
    { attrs : List (Html.Attribute msg)
    , styles : List (Html.Attribute msg)
    }

-- Make an element a drop target
droppable : Config id msg -> State id -> id -> Attributes msg

-- Check if a draggable is currently over this drop target
isOver : id -> State id -> Bool

-- Highlight styles for when a draggable is over the drop zone
activeStyles : State id -> id -> List (Html.Attribute msg)

-- Associate arbitrary data with a droppable
withData : List ( String, String ) -> Attributes msg -> Attributes msg
```

DnD.Sortable

```
-- High-level sortable list configuration
type alias Config id msg =
    { items : List id
    , onReorder : List id -> msg
    , toId : id -> String
    , viewItem : id -> SortableItemState -> Html msg
    , layout : Layout
    }

-- Layout direction for sorting
type Layout
    = Vertical
    | Horizontal
    | Grid { columns : Int }

-- State of each item during a sort operation
type alias SortableItemState =
    { isDragging : Bool
    , isOver : Bool
    , transform : Maybe Delta
    , transition : Maybe String
    }

-- Render a sortable list
view : Config id msg -> DnD.State id -> Html msg

-- Compute the reordered list after a drag end
reorder : DragEnd id -> List id -> List id
```

```
-- Compute the reordered list for cross-container moves
moveBetween :
    { from : List id
    , to : List id
    , active : id
    , over : id
    }
    -> { from : List id, to : List id }
```

DnD.Sensor

```
-- A sensor detects user input and translates it to drag operations
type Sensor id

-- Pointer sensor: handles mouse, touch, and pen via pointer events
pointer : PointerConfig -> Sensor id

type alias PointerConfig =
    { activationDistance : Float  -- pixels of movement before drag starts
    , activationDelay : Int      -- milliseconds to hold before drag starts (for
touch)
    }

-- Keyboard sensor: arrow keys to move, space/enter to pick up/drop
keyboard : KeyboardConfig -> Sensor id

type alias KeyboardConfig =
    { scrollSpeed : Float       -- pixels per key press to scroll
    , moveStep : Float          -- pixels per arrow key press to move item
    }

-- Default sensors: pointer with 5px activation, keyboard with defaults
defaults : List (Sensor id)
```

DnD.Collision

```
-- Strategy for detecting which drop target the dragged item is over
type CollisionDetection id

-- The drop target whose center is closest to the pointer
closestCenter : CollisionDetection id

-- The drop target whose corners are closest to the dragged item's corners
closestCorners : CollisionDetection id

-- The first drop target that intersects with the dragged item's rect
rectIntersection : CollisionDetection id
```

```
-- The drop target with the most overlap area with the dragged item
mostOverlap : CollisionDetection id

-- Custom collision detection
custom : (DragItem id -> List (DropTarget id) -> Maybe (DropTarget id)) ->
CollisionDetection id
```

DnD.Modifier

```
-- Modifiers transform the drag position before it's applied
type Modifier id

-- Lock movement to horizontal axis only
restrictToHorizontalAxis : Modifier id

-- Lock movement to vertical axis only
restrictToVerticalAxis : Modifier id

-- Restrict dragging to within the parent container bounds
restrictToParentElement : Modifier id

-- Restrict dragging to within a specific element's bounds
restrictToElement : String -> Modifier id

-- Snap to a grid
snapToGrid : { x : Float, y : Float } -> Modifier id

-- Snap to the center of drop targets
snapToCenter : Modifier id

-- Custom modifier
custom : (Delta -> DragItem id -> Delta) -> Modifier id
```

DnD.Announcements

```
-- ARIA live region announcement configuration
type Announcements id

-- Default English announcements
defaults : (id -> String) -> Announcements id

-- Custom announcements for each drag lifecycle event
custom :
    { onDragStart : DragItem id -> String
    , onDragOver : DragItem id -> DropTarget id -> String
    , onDragEnd : DragItem id -> Maybe (DropTarget id) -> String
    , onDragCancel : DragItem id -> String
    }
    -> Announcements id
```

```
-- Render the ARIA live region (place once in your view, typically near the root)
viewLiveRegion : DnD.State id -> Html msg
```

DnD.AutoScroll

```
-- Auto-scroll configuration for when dragging near container edges
type alias AutoScrollConfig =
    { enabled : Bool
    , threshold : Float   -- pixels from edge to start scrolling
    , speed : Float       -- pixels per frame to scroll
    , acceleration : Float -- multiplier as cursor gets closer to edge
    }

-- Default auto-scroll settings
defaults : AutoScrollConfig

-- Disabled auto-scroll
disabled : AutoScrollConfig
```

Example: Sortable Todo List

```
module Main exposing (main)

import Browser
import DnD
import DnD.Sortable as Sortable
import DnD.Sensor as Sensor
import DnD.Collision as Collision
import DnD.Announcements as Announcements
import Html exposing (Html, div, text, li, ul)
import Html.Attributes exposing (class, style)


type alias Model =
    { todos : List String
    , dnd : DnD.State String
    }


type Msg
    = DragStarted (DnD.DragStart String)
    | DraggedOver (DnD.DragOver String)
    | DragEnded (DnD.DragEnd String)
    | DragCancelled
    | TodosReordered (List String)


init : Model
```

```
init =
    { todos = [ "Buy groceries", "Walk the dog", "Write code", "Read a book" ]
    , dnd = DnD.init
    }


dndConfig : DnD.Config String Msg
dndConfig =
    { onDragStart = DragStarted
    , onDragOver = DraggedOver
    , onDragEnd = DragEnded
    , onDragCancel = DragCancelled
    , sensors = Sensor.defaults
    , collisionDetection = Collision.closestCenter
    , modifiers = []
    , announcements = Announcements.defaults identity
    , autoScroll = DnD.AutoScroll.defaults
    }


update : Msg -> Model -> ( Model, Cmd Msg )
update msg model =
    case msg of
        DragStarted _ ->
            ( model, Cmd.none )

        DraggedOver _ ->
            ( model, Cmd.none )

        DragEnded dragEnd ->
            ( { model | todos = Sortable.reorder dragEnd model.todos }
            , Cmd.none
            )

        DragCancelled ->
            ( model, Cmd.none )

        TodosReordered newTodos ->
            ( { model | todos = newTodos }, Cmd.none )


view : Model -> Html Msg
view model =
    div [ class "todo-list" ]
        [ Sortable.view
            { items = model.todos
            , onReorder = TodosReordered
            , toId = identity
            , viewItem = viewTodo
            , layout = Sortable.Vertical
            }
            model.dnd
```

```
        , Announcements.viewLiveRegion model.dnd
        ]


viewTodo : String -> Sortable.SortableItemState -> Html Msg
viewTodo todo state =
    li
        [ class "todo-item"
        , class (if state.isDragging then "dragging" else "")
        , class (if state.isOver then "over" else "")
        ]
        [ text todo ]


subscriptions : Model -> Sub Msg
subscriptions model =
    DnD.subscriptions dndConfig model.dnd


main : Program () Model Msg
main =
    Browser.element
        { init = \_ -> ( init, Cmd.none )
        , update = update
        , view = view
        , subscriptions = subscriptions
        }
```

Example: Kanban Board (Cross-Container)

```
module Kanban exposing (main)

import Browser
import DnD
import DnD.Draggable as Draggable
import DnD.Droppable as Droppable
import DnD.Sortable as Sortable
import DnD.Collision as Collision
import Html exposing (Html, div, text, h3)
import Html.Attributes exposing (class)
import Dict exposing (Dict)


type alias CardId = String

type alias ColumnId = String

type alias Model =
    { columns : Dict ColumnId (List CardId)
    , cards : Dict CardId String
```

```
    , dnd : DnD.State CardId
    }


type Msg
    = DndMsg (DnD.DragStart CardId)
    | DndOver (DnD.DragOver CardId)
    | DndEnd (DnD.DragEnd CardId)
    | DndCancel


update : Msg -> Model -> ( Model, Cmd Msg )
update msg model =
    case msg of
        DndEnd dragEnd ->
            -- Determine source and target columns from droppable data
            -- Reorder within column or move between columns
            ( applyMove dragEnd model, Cmd.none )

        _ ->
            ( model, Cmd.none )


applyMove : DnD.DragEnd CardId -> Model -> Model
applyMove dragEnd model =
    -- Implementation determines source/target columns and performs the move
    model


viewColumn : DnD.State CardId -> ColumnId -> List CardId -> Dict CardId String ->
Html Msg
viewColumn dndState columnId cardIds cards =
    let
        dropAttrs = Droppable.droppable dndConfig dndState columnId
    in
    div (class "column" :: dropAttrs.attrs ++ dropAttrs.styles)
        [ h3 [] [ text columnId ]
        , div [ class "card-list" ]
            (List.map (\cid -> viewCard dndState cid (Dict.get cid cards)) cardIds)
        ]


viewCard : DnD.State CardId -> CardId -> Maybe String -> Html Msg
viewCard dndState cardId maybeTitle =
    let
        dragAttrs = Draggable.draggable dndConfig dndState cardId
    in
    div (class "card" :: dragAttrs.listeners ++ dragAttrs.styles ++ dragAttrs.attrs)
        [ text (Maybe.withDefault "" maybeTitle) ]
```

Implementation Notes

Pointer Event Strategy

The library uses pointer events (`pointerdown` , `pointermove` , `pointerup`) rather than separate mouse/touch event handling. This provides:

- Unified handling across mouse, touch, and pen input
- `setPointerCapture` for reliable tracking even when pointer leaves the element
- No need for touch-delay hacks or polyfills

Pointer events are subscribed via `Browser.Events` during an active drag, and only `pointerdown` listeners are attached to draggable elements when idle.

Animation Strategy

During sortable reordering, displaced items animate to their new positions using CSS transitions (`transform` with `transition`). The dragged item follows the pointer with no transition (immediate response). On drop, the dragged item animates to its final position.

Keyboard Interaction

Following WAI-ARIA best practices:

1. Tab to focus a draggable item
2. Space/Enter to pick up (announces "Item grabbed, current position 1 of 5")
3. Arrow keys to move (announces "Item moved, new position 2 of 5")
4. Space/Enter to drop (announces "Item dropped, final position 2 of 5")
5. Escape to cancel (announces "Reorder cancelled, item returned to position 1 of 5")

FFI Requirements

Minimal FFI needed:

- `getBoundingClientRect` for element measurement (may already be in `Browser.Dom`)
- `setPointerCapture` / `releasePointerCapture` for reliable pointer tracking
- `scrollBy` for auto-scroll behavior
- ARIA live region can be pure HTML with `aria-live="assertive"`

Performance Considerations

- Only measure rects on drag start, cache during drag
- Use `requestAnimationFrame` for smooth dragging (via `Browser.Events.onAnimationFrame`)
- Minimize DOM reads during drag (use cached positions + deltas)
- CSS `will-change: transform` on dragged items

Line Estimates

Module	Lines	Status
DnD	250	v1 implemented
DnD.Draggable	120	v1 implemented
DnD.Droppable	100	v1 implemented

DnD.Sortable	350	v1 implemented
DnD.Sensor	200	v1 implemented
DnD.Collision	180	v1 implemented
DnD.Modifier	150	v1 implemented
DnD.Announcements	100	v1 implemented
DnD.AutoScroll	120	v1 implemented
FFI (external/*.js)	150	v1 implemented
v1 Total	**~1720**	
DnD.Tree	450	v2 planned
DnD.Kanban	400	v2 planned
DnD.Grid	500	v2 planned
DnD.MultiSelect	350	v2 planned
DnD.Handle	80	v2 planned
DnD.Animation	300	v2 planned
DnD.Hitbox	200	v2 planned
DnD.Monitor	150	v2 planned
DnD.Operation	120	v2 planned
DnD.FileDrop	250	v2 planned
DnD.Virtualized	300	v2 planned
DnD.History	200	v2 planned
DnD.Testing	250	v2 planned
FFI v2 (external/*.js)	200	v2 planned
v2 Total	**~3550**	
Grand Total	**~5270**	

Elm Equivalent Reference

Existing Elm Packages

Package	Author	Description
annaghi/dnd-list 6.0.1	annaghi	Sortable list via drag-and-drop with `DnDList` and `DnDList.Groups` modules; mouse-only, TEA-native state management

norpan/elm-html5-drag-drop 3.1.4	norpan	Wrapper around the HTML5 Drag and Drop API via `Html5.DragDrop` module; supports typed drag data between draggable and droppable elements

Key Differences from Canopy Plan

- Canopy uses **pointer events** instead of mouse events (annaghi/dnd-list) or HTML5 DnD API (norpan/elm-html5-drag-drop), providing unified mouse/touch/pen support without polyfills
- **Keyboard accessibility** is built in with arrow key navigation, space/enter pick-up/drop, and ARIA live region announcements, which neither Elm package provides
- **Cross-container dragging** (kanban board style) is a first-class feature, while annaghi/dnd-list's `DnDList.Groups` provides limited support and norpan/elm-html5-drag-drop leaves it to the user
- **Collision detection strategies** are pluggable (`closestCenter` , `rectIntersection` , `mostOverlap` , custom), inspired by dnd-kit, while Elm packages use fixed detection logic
- **Modifiers** for axis locking, grid snapping, and bounds restriction are composable, which no Elm package offers
- **Auto-scroll** near container edges during drag is built in, absent from both Elm packages
- **Drag overlays** allow custom drag preview rendering without DOM cloning

Design Inspirations

- Adopt annaghi/dnd-list's TEA-native state management pattern with opaque `State` type and explicit model integration
- Adopt annaghi/dnd-list's high-level `Sortable` API for the common case of reorderable lists
- Avoid norpan/elm-html5-drag-drop's reliance on the HTML5 DnD API, which is inconsistent across browsers and non-functional on touch devices
- Follow dnd-kit's layered architecture (sensors, collision detection, modifiers) rather than Elm packages' monolithic approach

---

Open Questions (Resolved)

1. **Ghost element vs CSS transform**: CSS transforms for v1 (simpler, TEA-friendly). Custom drag overlays via `viewOverlay` handle visual customization without DOM cloning. This is the approach dnd-kit uses and it works well.

2. **Cross-window dragging**: Out of scope. Pointer events cannot do this. If needed later, a separate `DnD.Native` adapter could wrap the HTML5 DnD API for this single use case.

3. **File drop from OS**: Yes, via `DnD.FileDrop` module (v2). It's a different API (`dragenter` / `dragover` / `drop` with `DataTransfer`) but belongs in this package since users expect unified DnD. Keeps the file-specific logic isolated.

4. **Integration with canopy/virtual-dom keyed**: Yes. `DnD.Sortable.view` uses `Html.Keyed.node` automatically. The `toId` function in the config provides the key. `DnD.Virtualized` (v2) extends this to windowed lists.

5. **Multi-select drag**: v2 via `DnD.MultiSelect` module. Complex but essential for file managers, image galleries, kanban multi-card moves. Includes keyboard multi-select (Shift+click, Ctrl+click).

6. **Drag handles**: v2 via `DnD.Handle` module. Separates the activator element from the draggable container. Simple API -- just returns attributes for the handle sub-element.

V2 Advanced Modules

Everything below represents the features that will make canopy/dnd the best DnD library available. These build on the v1 core primitives.

DnD.Tree -- Tree Drag-and-Drop

Drag items within and between tree structures. Supports file explorers, nested navigation menus, org charts, and any hierarchical data. Inspired by Pragmatic DnD's tree adapter and React Aria's tree DnD.

No existing DnD library handles trees well. Pragmatic DnD has a tree example but no dedicated API. dnd-kit requires extensive custom code. This is our opportunity to be definitively best-in-class.

```
-- A tree node with children
type alias TreeNode id =
    { id : id
    , children : List (TreeNode id)
    , isExpanded : Bool
    }

-- Configuration for tree drag-and-drop
type alias Config id msg =
    { tree : List (TreeNode id)
    , onReorder : List (TreeNode id) -> msg
    , toId : id -> String
    , viewNode : id -> TreeNodeState -> List (Html msg) -> Html msg
    , maxDepth : Maybe Int              -- Maximum nesting depth (Nothing =
unlimited)
    , canDropInto : id -> id -> Bool    -- Can source drop as child of target?
    , canDropBefore : id -> id -> Bool  -- Can source drop before target?
    , expandOnHoverDelay : Int          -- ms before expanding a collapsed node on
hover (default: 500)
    , indentWidth : Float               -- px per depth level (default: 24)
    }

-- Visual state for each tree node during drag
type alias TreeNodeState =
    { isDragging : Bool
    , isOver : Bool
    , dropPosition : Maybe DropPosition
    , depth : Int
    , isExpanded : Bool
    , isAncestorOfDragged : Bool  -- True if this node is an ancestor of the dragged
item
    }

-- Where the drop indicator shows relative to a node
type DropPosition
    = Before          -- Drop as sibling before this node
    | After           -- Drop as sibling after this node
    | Inside          -- Drop as child of this node (append to children)
```

```
-- Render the tree with drag-and-drop enabled
view : Config id msg -> DnD.State id -> Html msg

-- Flatten tree to list for collision detection (used internally, exposed for
advanced use)
flatten : List (TreeNode id) -> List { id : id, depth : Int, parentId : Maybe id }

-- Apply a tree move operation
applyMove :
    { active : id
    , target : id
    , position : DropPosition
    }
    -> List (TreeNode id)
    -> List (TreeNode id)

-- Validate a move doesn't create a cycle (can't drop parent into its own child)
isValidMove : id -> id -> List (TreeNode id) -> Bool

-- Expand a node (typically called on hover during drag)
expandNode : id -> List (TreeNode id) -> List (TreeNode id)

-- Collapse a node
collapseNode : id -> List (TreeNode id) -> List (TreeNode id)
```

Key behaviors:

- Hovering over a collapsed node for `expandOnHoverDelay` ms expands it, revealing children as drop targets
- Drop indicators show before/after/inside based on cursor position within the node:
  - Top 25%: drop before
  - Bottom 25%: drop after
  - Middle 50%: drop as child (inside)
- Dragging a parent drags all its children with it
- Cannot drop a node into its own descendants (cycle prevention)
- `maxDepth` prevents excessive nesting
- Keyboard: arrow keys navigate the tree, Space/Enter to grab, arrow keys to move between positions, Tab to cycle through before/after/inside

DnD.Kanban -- Kanban Board

High-level API for kanban boards with sortable columns and cross-column card movement. This is the #1 use case for DnD libraries (Trello, Jira, Notion) and we provide it as a first-class module rather than leaving users to assemble it from primitives.

```
-- Kanban board configuration
type alias Config columnId cardId msg =
    { columns : List (Column columnId cardId)
    , onCardMove : CardMove columnId cardId -> msg
    , onColumnReorder : List columnId -> msg
    , viewCard : cardId -> CardState -> Html msg
    , viewColumn : columnId -> ColumnState -> List (Html msg) -> Html msg
```

```
    , viewEmptyColumn : columnId -> Html msg
    , cardCollision : DnD.Collision.CollisionDetection cardId
    , columnCollision : DnD.Collision.CollisionDetection columnId
    , canDrop : columnId -> cardId -> Bool  -- WIP limits, permission checks
    , columnsDraggable : Bool               -- Can columns themselves be reordered?
    }

type alias Column columnId cardId =
    { id : columnId
    , cards : List cardId
    }

-- Result of a card move operation
type alias CardMove columnId cardId =
    { cardId : cardId
    , fromColumn : columnId
    , toColumn : columnId
    , fromIndex : Int
    , toIndex : Int
    }

-- Visual state for cards during drag
type alias CardState =
    { isDragging : Bool
    , isOver : Bool
    , isInActiveColumn : Bool  -- Column currently being hovered
    }

-- Visual state for columns during drag
type alias ColumnState =
    { isOver : Bool              -- A card is hovering over this column
    , isDragging : Bool          -- This column itself is being dragged
    , cardCount : Int
    , isDropDisabled : Bool      -- canDrop returned False
    }

-- Kanban-specific state (wraps DnD.State)
type State columnId cardId

-- Initialize
init : State columnId cardId

-- Render the full kanban board
view : Config columnId cardId msg -> State columnId cardId -> Html msg

-- Apply a card move to the column data
applyCardMove : CardMove columnId cardId -> List (Column columnId cardId) -> List
(Column columnId cardId)

-- Subscriptions
subscriptions : Config columnId cardId msg -> State columnId cardId -> Sub msg
```

Key behaviors:

- Cards are sortable within their column (vertical sort)
- Cards can be dragged between columns (cross-container)
- Empty columns are valid drop targets (shows empty state)
- `canDrop` enables WIP limits ("this column accepts max 5 cards")
- Column headers can optionally be dragged to reorder columns (horizontal sort)
- Keyboard: Tab between columns, arrow keys within column, Ctrl+arrow to move across columns
- Announcements: "Card 'Fix bug' moved from 'In Progress' column to 'Done' column, position 3 of 7"

DnD.Grid -- Dashboard Grid Layout

Drag-and-resize grid tiles for dashboard builders, like Gridstack.js or react-grid-layout, but TEA-native and accessible. This is a use case where no functional/declarative library provides a good solution.

```
-- Grid layout configuration
type alias Config id msg =
    { items : List (GridItem id)
    , columns : Int                    -- Number of grid columns (e.g., 12)
    , rowHeight : Float                -- Height of each row in pixels
    , gap : Float                      -- Gap between items in pixels
    , onLayoutChange : List (GridItem id) -> msg
    , viewItem : id -> GridItemState -> Html msg
    , compactMode : CompactMode        -- How items collapse when space opens
    , resizable : Bool                 -- Can items be resized?
    , draggable : Bool                 -- Can items be repositioned?
    , bounds : GridBounds              -- Constraintsfor the grid
    }

type alias GridItem id =
    { id : id
    , x : Int           -- Column position (0-based)
    , y : Int           -- Row position (0-based)
    , width : Int        -- Width in grid units
    , height : Int       -- Height in grid units
    , minWidth : Int     -- Minimum width (default: 1)
    , minHeight : Int    -- Minimum height (default: 1)
    , maxWidth : Int     -- Maximum width (default: columns)
    , maxHeight : Int    -- Maximum height (default: Infinity)
    , isStatic : Bool    -- Cannot be moved or resized
    }

type CompactMode
    = CompactVertical     -- Items float up to fill vertical gaps
    | CompactHorizontal   -- Items float left to fill horizontal gaps
    | CompactNone         -- Items stay where placed

type GridBounds
    = Unbounded                   -- Grid grows infinitely
    | BoundedRows Int             -- Maximum number of rows
    | BoundedToContainer          -- Grid height matches container
```

```
type alias GridItemState =
    { isDragging : Bool
    , isResizing : Bool
    , isPlaceholder : Bool    -- Ghost showing where item will land
    , isStatic : Bool
    }

-- Resize handle positions
type ResizeHandle
    = ResizeSE    -- Bottom-right (default)
    | ResizeSW    -- Bottom-left
    | ResizeNE    -- Top-right
    | ResizeNW    -- Top-left
    | ResizeS     -- Bottom edge
    | ResizeE     -- Right edge
    | ResizeW     -- Left edge
    | ResizeN     -- Top edge

-- Grid state
type State id

init : State id

-- Render the grid layout
view : Config id msg -> State id -> Html msg

-- Compute layout after a drag or resize
compact : CompactMode -> Int -> List (GridItem id) -> List (GridItem id)

-- Check for collisions between grid items
hasCollision : GridItem id -> List (GridItem id) -> Bool

-- Resolve collisions by pushing items down
resolveCollisions : GridItem id -> List (GridItem id) -> List (GridItem id)

-- Generate responsive breakpoint layouts
type alias ResponsiveConfig id =
    { breakpoints : List ( String, Int )  -- ("lg", 1200), ("md", 996), ("sm", 768)
    , layouts : Dict String (List (GridItem id))
    , currentBreakpoint : String
    }

-- Subscriptions (window resize for responsive)
subscriptions : Config id msg -> State id -> Sub msg
```

Key behaviors:

- Drag items to reposition on the grid, snap to grid cells
- Resize items by dragging handles on edges/corners
- Collision resolution: when an item is placed, overlapping items push down
- Compact modes prevent gaps (items "float" up/left)
- Responsive breakpoints for different screen sizes

- Keyboard: arrow keys to move by one grid cell, Shift+arrow to resize
- Static items cannot be moved/resized but others flow around them
- Placeholder ghost shows where the item will land during drag

DnD.MultiSelect -- Multi-Item Drag

Select and drag multiple items simultaneously. Essential for file managers, image galleries, email clients, and anywhere users work with collections.

```
-- Multi-select state (wraps DnD.State, adds selection tracking)
type State id

init : State id

-- Selection operations
select : id -> State id -> State id
deselect : id -> State id -> State id
toggleSelect : id -> State id -> State id
selectRange : id -> id -> List id -> State id -> State id  -- Shift+click range
selectAll : List id -> State id -> State id
clearSelection : State id -> State id

-- Query selection
selectedItems : State id -> Set id
isSelected : id -> State id -> Bool
selectionCount : State id -> Int

-- Configuration
type alias Config id msg =
    { onDragStart : MultiDragStart id -> msg
    , onDragOver : MultiDragOver id -> msg
    , onDragEnd : MultiDragEnd id -> msg
    , onDragCancel : msg
    , onSelectionChange : Set id -> msg
    , sensors : List (DnD.Sensor id)
    , collisionDetection : DnD.Collision.CollisionDetection id
    }

type alias MultiDragStart id =
    { active : DnD.DragItem id
    , selected : Set id       -- All items being dragged (including active)
    }

type alias MultiDragOver id =
    { active : DnD.DragItem id
    , selected : Set id
    , over : Maybe (DnD.DropTarget id)
    , delta : DnD.Delta
    }

type alias MultiDragEnd id =
    { active : DnD.DragItem id
```

```
    , selected : Set id
    , over : Maybe (DnD.DropTarget id)
    }

-- Attributes for multi-selectable + draggable items
multiDraggable : Config id msg -> State id -> id -> DnD.Draggable.Attributes msg

-- View a count badge on the drag overlay ("3 items")
viewDragCount : State id -> Html msg

-- Apply multi-item reorder (moves all selected items to the drop position)
reorderMulti : MultiDragEnd id -> List id -> List id

-- Apply multi-item cross-container move
moveMultiBetween :
    { from : List id
    , to : List id
    , selected : Set id
    , over : id
    }
    -> { from : List id, to : List id }
```

Key behaviors:

- Click to select, Ctrl/Cmd+click to toggle, Shift+click for range select
- Dragging any selected item drags all selected items
- Drag overlay shows count badge ("Dragging 4 items")
- Non-selected items shift to fill gaps as the group moves
- Drop inserts all selected items at the target position, preserving their relative order
- Keyboard: Space to toggle selection, Ctrl+A for select all, then standard DnD keyboard

DnD.Handle -- Drag Handles

Separate the drag activation zone from the draggable container. The grip icon (or any sub-element) starts the drag, but the entire container moves.

```
-- Create handle attributes for the sub-element that initiates drag
handle : DnD.Config id msg -> DnD.State id -> id -> DnD.Draggable.Attributes msg

-- Create container attributes for the element that moves (no listeners, just
styles)
container : DnD.State id -> id -> { styles : List (Html.Attribute msg), attrs : List
(Html.Attribute msg) }
```

Usage:

```
viewItem : id -> Html Msg
viewItem itemId =
    let
        handleAttrs = Handle.handle dndConfig dndState itemId
        containerAttrs = Handle.container dndState itemId
    in
```

```
    div (class "item" :: containerAttrs.styles ++ containerAttrs.attrs)
        [ span (class "grip-icon" :: handleAttrs.listeners ++ handleAttrs.attrs)
            [ text "⠿" ]
        , span [] [ text "Item content" ]
        ]
```

DnD.Animation -- Spring Physics

The animation system that made react-beautiful-dnd beloved. Every DnD library without spring physics feels "dead." Our animations must feel physical -- items have weight, momentum, and respond naturally.

```
-- Spring configuration
type alias SpringConfig =
    { stiffness : Float    -- Spring tension (default: 300)
    , damping : Float      -- Friction (default: 25)
    , mass : Float         -- Weight of the item (default: 1)
    }

-- Predefined spring presets
gentle : SpringConfig      -- Slow, soft movement (stiffness: 120, damping: 14)
wobbly : SpringConfig      -- Bouncy, playful (stiffness: 180, damping: 12)
stiff : SpringConfig       -- Quick, precise (stiffness: 400, damping: 30)
snappy : SpringConfig      -- Fast with slight overshoot (stiffness: 300, damping:
25)

-- Drop animation: animates the dragged item from release point to final position
type alias DropAnimation =
    { spring : SpringConfig
    , enabled : Bool
    }

defaultDropAnimation : DropAnimation

-- Displacement animation: how items move out of the way during drag
type alias DisplacementAnimation =
    { curve : AnimationCurve
    , duration : Int         -- ms (default: 250)
    }

type AnimationCurve
    = Spring SpringConfig
    | CubicBezier Float Float Float Float   -- CSS cubic-bezier values
    | Custom String                          -- Raw CSS timing function

-- react-beautiful-dnd's displacement curve: warm-up, fast phase, long tail
-- This specific curve lets users read text on items as they move
rbd : AnimationCurve  -- cubic-bezier(0.2, 0, 0, 1)

-- Generate CSS transition string for an animation
transitionString : DisplacementAnimation -> String
```

```
-- Compute spring position at a given time (for requestAnimationFrame-driven
animation)
springPosition : SpringConfig -> { from : Float, to : Float, velocity : Float, time
: Float } -> { position : Float, velocity : Float, isSettled : Bool }

-- Scale-up effect when picking up an item (subtle, like lifting off the page)
type alias LiftEffect =
    { scale : Float         -- Scale factor on pickup (default: 1.02)
    , shadow : String       -- Box shadow CSS (default: "0 5px 15px
rgba(0,0,0,0.15)")
    , rotation : Float      -- Slight rotation in degrees (default: 0)
    , duration : Int        -- ms for the lift transition (default: 200)
    }

defaultLiftEffect : LiftEffect

-- Apply lift effect styles to the dragged item
liftStyles : LiftEffect -> DnD.State id -> id -> List (Html.Attribute msg)
```

Key behaviors:

- Drop animation: when the user releases, the item doesn't teleport -- it springs to its final position with physics-based deceleration
- Displacement: when items move to make room, they follow a curve designed for readability (fast enough to feel responsive, slow enough to track visually)
- Lift effect: subtle scale-up and shadow increase on pickup creates a "peeling off the page" feel
- All animations use CSS transitions/transforms for GPU acceleration
- Spring calculations available for JS-driven animations via requestAnimationFrame when CSS isn't sufficient

DnD.Hitbox -- Edge Proximity Detection

Detect which edge of a drop target the cursor is closest to. This is critical for sortable lists (should the item go before or after?) and trees (before/after/inside). Inspired by Pragmatic DnD's `@atlaskit/pragmatic-drag-and-drop-hitbox` package.

```
-- Which edge of the target is the cursor closest to
type Edge
    = Top
    | Bottom
    | Left
    | Right

-- Detect the closest edge given a point and a rect
closestEdge : DnD.Point -> DnD.Rect -> Edge

-- Detect closest edge with exclusions (e.g., in a vertical list, only care about
top/bottom)
closestEdgeFiltered : List Edge -> DnD.Point -> DnD.Rect -> Maybe Edge

-- Compute which zone of the target the cursor is in (for tree nodes)
type Zone
```

```
    = TopQuarter      -- Top 25%: drop before
    | BottomQuarter   -- Bottom 25%: drop after
    | Center          -- Middle 50%: drop inside

zone : DnD.Point -> DnD.Rect -> Zone

-- Custom zone thresholds
zoneWithThresholds : { edgeThreshold : Float } -> DnD.Point -> DnD.Rect -> Zone

-- Attach edge data to drop targets (store in droppable's data dict)
attachClosestEdge : DnD.Point -> DnD.Rect -> List ( String, String )

-- Extract edge from droppable data
extractClosestEdge : Dict String String -> Maybe Edge
```

DnD.Monitor -- Global Drag Observers

Observe drag operations globally without being tied to a specific draggable or droppable element. Inspired by Pragmatic DnD's `monitorForElements`. Useful for:

- Analytics (track drag frequency)
- Global side effects (dim the background during drag)
- Cross-component communication
- Debug logging

```
-- A monitor subscription that fires for all drag events
type alias MonitorConfig msg =
    { onDragStart : Maybe (DnD.DragStart id -> msg)
    , onDragOver : Maybe (DnD.DragOver id -> msg)
    , onDragEnd : Maybe (DnD.DragEnd id -> msg)
    , onDragCancel : Maybe msg
    }

-- Subscribe to global drag events
monitor : MonitorConfig msg -> DnD.State id -> Sub msg

-- Check if ANY drag is currently in progress (useful for dimming background)
isAnyDragActive : DnD.State id -> Bool

-- Get the type/category of the current drag (from item data)
dragType : DnD.State id -> Maybe String
```

DnD.Operation -- Drag Operation Types

Different semantics for what a drag means: move (relocate), copy (duplicate), or link (create reference). Visual cursors and drop behavior change accordingly. Like the `effectAllowed` / `dropEffect` from HTML5 DnD but properly implemented.

```
type Operation
    = Move      -- Default: relocate item to new position
    | Copy      -- Duplicate item at new position, original stays
```

```
    | Link      -- Create a reference/shortcut at new position

-- Determine operation based on modifier keys held during drag
fromModifierKeys : { ctrl : Bool, alt : Bool, shift : Bool } -> Operation

-- Default mapping: no modifier = Move, Ctrl = Copy, Alt = Link
defaultOperationMap : { ctrl : Bool, alt : Bool, shift : Bool } -> Operation

-- Get cursor style for current operation
cursorForOperation : Operation -> String

-- Attach operation to drag state
withOperation : Operation -> DnD.Config id msg -> DnD.Config id msg
```

DnD.FileDrop -- OS File Drop Zones

Handle files dragged from the operating system's file manager into the browser. Uses the HTML5 `dragenter` / `dragover` / `drop` events with `DataTransfer` API, completely separate from pointer-event-based DnD but sharing the same visual language.

```
-- File drop zone configuration
type alias Config msg =
    { onDrop : List File -> msg
    , onDragEnter : msg
    , onDragLeave : msg
    , accept : List String               -- MIME types ("image/*",
"application/pdf")
    , maxFiles : Maybe Int               -- Maximum number of files (Nothing =
unlimited)
    , maxFileSize : Maybe Int            -- Maximum file size in bytes
    , disabled : Bool
    }

-- File metadata (decoded from DataTransfer)
type alias File =
    { name : String
    , size : Int
    , mimeType : String
    , lastModified : Int                 -- Unix timestamp ms
    }

-- State for tracking hover
type State

init : State

-- Is the user hovering files over the drop zone?
isActive : State -> Bool

-- Attributes for the drop zone element
dropZone : Config msg -> State -> { attrs : List (Html.Attribute msg), styles : List
```

```
(Html.Attribute msg) }

-- Validate files against config constraints
validate : Config msg -> List File -> { accepted : List File, rejected : List (
File, RejectionReason ) }

type RejectionReason
    = InvalidMimeType
    | FileTooLarge
    | TooManyFiles

-- Read file contents (returns Cmd that produces file data)
readAsText : File -> (String -> msg) -> Cmd msg
readAsDataUrl : File -> (String -> msg) -> Cmd msg
readAsBytes : File -> (Bytes -> msg) -> Cmd msg

-- Combine with sortable: a list where you can both reorder existing items
-- AND drop new files from OS. Useful for image galleries.
type alias SortableDropZoneConfig id msg =
    { sortable : DnD.Sortable.Config id msg
    , fileDrop : Config msg
    }
```

DnD.Virtualized -- Virtualized List Integration

Support for drag-and-drop in virtualized/windowed lists with thousands of items. The fundamental challenge: items not currently rendered in the DOM can't be measured or used as drop targets. This module solves that.

```
-- Virtualized sortable configuration
type alias Config id msg =
    { items : List id
    , onReorder : List id -> msg
    , toId : id -> String
    , viewItem : id -> Int -> DnD.Sortable.SortableItemState -> Html msg
    , layout : DnD.Sortable.Layout
    , itemHeight : ItemHeight id         -- How to determine item heights
    , overscan : Int                     -- Extra items to render above/below
visible area (default: 5)
    , containerHeight : Float            -- Visible container height in pixels
    }

type ItemHeight id
    = FixedHeight Float                  -- All items same height
    | VariableHeight (id -> Float)       -- Known variable heights
    | MeasuredHeight                     -- Measure on first render, cache

-- State includes scroll position and rendered range
type State id

init : Config id msg -> State id
```

```
-- View the virtualized sortable list
view : Config id msg -> DnD.State id -> State id -> Html msg

-- Handle scroll events to update visible range
onScroll : (State id -> msg) -> Html.Attribute msg

-- Get the currently visible range
visibleRange : State id -> { start : Int, end : Int }

-- Scroll to a specific item (useful for keyboard DnD)
scrollToItem : id -> State id -> Cmd msg

-- Increased overscan during drag to prevent items from disappearing
dragOverscan : Int  -- default: 10, double the normal overscan during active drag
```

Key challenges solved:

- During drag, increase overscan so items don't vanish as the user scrolls
- Use cached heights for items not in DOM when calculating drop position
- Auto-scroll integrates with the virtual scroll position, not just container scroll
- Keyboard DnD scrolls the virtual list to reveal the target position

DnD.History -- Undo/Redo for Drag Operations

No DnD library provides built-in undo/redo. We do. Every drag operation is recorded and reversible. Critical for page builders and any tool where mistakes are costly.

```
-- History state parameterized by the model snapshot type
type State snapshot

-- Initialize with the current state as the first snapshot
init : snapshot -> State snapshot

-- Record a new state after a drag operation
push : snapshot -> State snapshot -> State snapshot

-- Undo the last operation (returns previous snapshot if available)
undo : State snapshot -> Maybe ( snapshot, State snapshot )

-- Redo a previously undone operation
redo : State snapshot -> Maybe ( snapshot, State snapshot )

-- Can we undo/redo? (for enabling/disabling UI buttons)
canUndo : State snapshot -> Bool
canRedo : State snapshot -> Bool

-- How many operations in history
undoCount : State snapshot -> Int
redoCount : State snapshot -> Int

-- Clear all history
clear : snapshot -> State snapshot
```

```
-- Set maximum history size (oldest entries dropped when exceeded)
withMaxSize : Int -> State snapshot -> State snapshot

-- Batch multiple operations into one undo step
-- (e.g., multi-select drag that moves 5 items = 1 undo step)
type alias Batch snapshot =
    { before : snapshot
    , after : snapshot
    }

pushBatch : Batch snapshot -> State snapshot -> State snapshot
```

DnD.Testing -- Test Utilities

Simulating drag in tests is notoriously difficult. We provide first-class test helpers so users can write reliable tests for their DnD interactions.

```
-- Simulate a complete drag-and-drop operation
simulateDrag :
    { from : id
    , to : id
    , state : DnD.State id
    , config : DnD.Config id msg
    }
    -> { messages : List msg, finalState : DnD.State id }

-- Simulate drag with intermediate positions
simulateDragPath :
    { from : id
    , through : List id     -- Intermediate hover targets
    , to : id
    , state : DnD.State id
    , config : DnD.Config id msg
    }
    -> { messages : List msg, finalState : DnD.State id }

-- Simulate a cancelled drag (pickup then Escape)
simulateDragCancel :
    { from : id
    , state : DnD.State id
    , config : DnD.Config id msg
    }
    -> { messages : List msg, finalState : DnD.State id }

-- Simulate keyboard drag
simulateKeyboardDrag :
    { from : id
    , moves : List KeyboardMove
    , state : DnD.State id
    , config : DnD.Config id msg
```

```
    }
    -> { messages : List msg, finalState : DnD.State id }

type KeyboardMove
    = ArrowUp
    | ArrowDown
    | ArrowLeft
    | ArrowRight
    | Drop          -- Space/Enter
    | Cancel        -- Escape

-- Create a mock DnD.State for testing views
mockState :
    { dragging : Maybe id
    , over : Maybe id
    , delta : DnD.Delta
    }
    -> DnD.State id

-- Assert that a list was correctly reordered
expectOrder : List id -> List id -> Expectation
```

Competitive Analysis: Where We Beat Every Library

Feature	Pragmatic DnD	dnd-kit	react-beautiful-dnd	SortableJS	canopy/dnd
Sortable lists	manual	yes	yes	yes	**yes**
Cross-container	manual	yes	yes	yes	**yes**
Kanban board	manual	manual	manual	manual	**first-class**
Tree DnD	example	manual	no	nested	**first-class**
Grid/dashboard layout	no	no	no	no	**first-class**
Multi-select drag	manual	manual	no	plugin	**first-class**
Spring physics	no	no	yes	no	**yes**
Keyboard DnD	building blocks	yes	yes	no	**yes**
Screen reader	building blocks	yes	yes	no	**yes**
Touch support	polyfill needed	yes	yes	fallback	**yes (pointer events)**

Virtualization	no	yes	addon	no	**first-class**
Drag handles	yes	yes	yes	yes	**yes**
File drop from OS	yes (adapter)	no	no	no	**yes**
Undo/redo	no	no	no	no	**yes**
Test utilities	no	no	no	no	**yes**
RTL support	partial	partial	partial	partial	**first-class**
Edge/hitbox detection	package	no	no	no	**built-in**
Drag operation types	no	no	no	no	**yes (move/copy/link)**
Framework-native	vanilla	React/Vue/Svelte	React only	vanilla	**Canopy (TEA-native)**
Bundle size (core)	~4.7kB	~12kB	~30kB	~12kB	**~5kB**

Implementation Phases

Phase 1: Core Polish (v1.1)

- Add dedicated Draggable and Droppable tests
- Improve animation with `DnD.Animation` (spring drops, displacement curves)
- Add `DnD.Handle` for drag handles
- Add `DnD.Hitbox` for edge detection
- Answer: keyed nodes in Sortable.view

Phase 2: Complex Patterns (v2.0)

- `DnD.Tree` -- tree drag-and-drop
- `DnD.Kanban` -- kanban board
- `DnD.MultiSelect` -- multi-item drag
- `DnD.Monitor` -- global observers
- `DnD.Operation` -- move/copy/link

Phase 3: Scale & Integration (v2.1)

- `DnD.Grid` -- dashboard grid layout
- `DnD.Virtualized` -- virtualized list integration
- `DnD.FileDrop` -- OS file drop zones
- `DnD.History` -- undo/redo
- `DnD.Testing` -- test utilities

Phase 4: Polish & Edge Cases (v2.2)

- RTL support across all modules
- Nested scroll container auto-scroll
- Zoom level compensation

- Performance benchmarks (target: 60fps with 1000+ items)
- Comprehensive examples for every module
- Full accessibility audit against WCAG 2.2

Example: Page Builder with Tree + Grid + FileDrop

The ultimate test of a DnD library -- a page builder where users can:

1. Drag components from a toolbox (copy operation)
2. Reorder components in a nested tree structure
3. Resize components on a grid
4. Drop images from the OS file manager

```
module PageBuilder exposing (main)

import DnD
import DnD.Tree as Tree
import DnD.Grid as Grid
import DnD.FileDrop as FileDrop
import DnD.Operation as Operation
import DnD.History as History
import DnD.Animation as Animation

type alias Model =
    { tree : List (Tree.TreeNode ComponentId)
    , grid : List (Grid.GridItem ComponentId)
    , dnd : DnD.State ComponentId
    , history : History.State LayoutSnapshot
    , fileDrop : FileDrop.State
    }

-- The toolbox uses Operation.Copy so dragging a component from
-- the toolbox creates a new instance, leaving the original in place.

-- The canvas uses Tree for structural hierarchy (sections > rows > columns >
components)
-- and Grid for visual positioning within each section.

-- FileDrop on image components allows replacing images by dropping from OS.

-- Every drag operation pushes to History, enabling Ctrl+Z/Ctrl+Y.
```

This is the kind of application that currently requires stitching together 3-4 different libraries in React. With canopy/dnd, it's one coherent package.